Universidad Nacional de Colombia
Facultad de Ciencias
Análisis Numérico I

Mateo Andrés Manosalva Amaris
Edgar Santiago Ochoa Quiroga
Sergio Alejandro Bello Torres

Ejercicio 1

Sea

$$A = \begin{bmatrix} 0 & -20 & 14 \\ -3 & 27 & 4 \\ -4 & 11 & 2 \end{bmatrix}$$

a) Aplique las transformaciones de Householder, para calcular $A = QR$.

Solución. Veamos $A = (a_1|a_2|a_3)$ donde cada a_i es el vector columna correspondiente a la matriz del enunciado. Primero tenemos que

$$\|a_1\| = \sqrt{0^2 + (-3)^2 + (-4)^2} = 5,$$

asi para construir nuestro reflector de Householder tomamos

$$v_1 = a_1 - \|a_1\| e_1 = \begin{bmatrix} 0 \\ -3 \\ -4 \end{bmatrix} - \begin{bmatrix} 5 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -5 \\ -3 \\ -4 \end{bmatrix}.$$

Luego tenemos que

$$H_1 = I_3 - 2\frac{v_1 \cdot v_1^T}{v_1^T \cdot v_1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \frac{2}{50}\begin{bmatrix} 25 & 15 & 20 \\ 15 & 9 & 12 \\ 20 & 12 & 16 \end{bmatrix} = \begin{bmatrix} 0 & -15/25 & -20/25 \\ -15/25 & 16/25 & -12/25 \\ -20/25 & -12/25 & 9/25 \end{bmatrix}.$$

Luego haciendo la multiplicacion con A tenemos

$$H_1 A = \begin{bmatrix} 5 & -25 & -4 \\ 0 & 24 & -34/5 \\ 0 & 7 & -62/5 \end{bmatrix}$$

Ahora nos concentraremos en el sub-bloque 2×2 de la matriz obtenida, es decir

$$\tilde{A} = \begin{bmatrix} 24 & -34/5 \\ 7 & -62/5 \end{bmatrix}$$

Reallizando el proceso previo, si $\tilde{A} = (\tilde{a}_1|\tilde{a}_2)$, tenemos que

$$\|\tilde{a}_1\| = \sqrt{24^2 + 7^2} = 25,$$

de esta manera

$$v_2 = \tilde{a}_1 - \|\tilde{a}_1\| e_1 = \begin{bmatrix} 24 \\ 7 \end{bmatrix} - \begin{bmatrix} 25 \\ 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 7 \end{bmatrix}$$

Si hacemos el reflector de Householder para $\tilde{A}$ obtenemos

$$\tilde{H}_2 = I_2 - 2\frac{v_2 \cdot v_2^T}{v_2^T \cdot v_2} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \frac{2}{50}\begin{bmatrix} 1 & -7 \\ -7 & 49 \end{bmatrix} = \begin{bmatrix} 24/25 & 7/25 \\ 7/25 & -24/25 \end{bmatrix}$$

De esta manera el reflector para A es

$$H_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 24/25 & 7/25 \\ 0 & 7/25 & -24/25 \end{bmatrix}$$

De esta manera tenemos que

$$H_2 H_1 A = \begin{bmatrix} 5 & -25 & -4 \\ 0 & 25 & -10 \\ 0 & 0 & 10 \end{bmatrix}$$

Note que esta matriz ya es triangular superior por lo que esa sera nuestra R, ahora como H_1 y H_2 son ortogonales y simetricos tenemos que:

$$A = H_1^T H_2^T R = H_1 H_2 R,$$

asi si realizamos la multiplicacion

$$Q = H_1 H_2 = \begin{bmatrix} 0 & -20/25 & 15/25 \\ -15/25 & 12/25 & 16/25 \\ -20/25 & -9/25 & -12/25 \end{bmatrix}.$$

Concluyendo asi que la factorizacion $A = QR$ es

$$\begin{bmatrix} 0 & -20 & 14 \\ -3 & 27 & 4 \\ -4 & 11 & 2 \end{bmatrix} = \begin{bmatrix} 0 & -20/25 & 15/25 \\ -15/25 & 12/25 & 16/25 \\ -20/25 & -9/25 & -12/25 \end{bmatrix} \begin{bmatrix} 5 & -25 & -4 \\ 0 & 25 & -10 \\ 0 & 0 & 10 \end{bmatrix}$$

b) Aplique el método de ortogonalización de Gram-Schmidt, para calcular $A = QR$.

Solución. Para este metodo tomamos nuevamente $A = (a_1|a_2|a_3)$, por medio de Gram-Schmidt conseguiremos vecotres ortonormales. Primero

$$q_1 = \frac{a_1}{\|a_1\|} = \begin{bmatrix} 0 \\ -3/5 \\ -4/5 \end{bmatrix}$$

Luego tomamos

$$v_1 = a_2 - (q_1^T a_2) q_1 = \begin{bmatrix} -20 \\ 27 \\ 11 \end{bmatrix} - (-25) \begin{bmatrix} 0 \\ -3/5 \\ -4/5 \end{bmatrix} = \begin{bmatrix} -20 \\ 12 \\ -9 \end{bmatrix}$$

Asi nuestro vector ortonormal es

$$q_2 = \frac{v_1}{\|v_1\|} = \begin{bmatrix} -20/25 \\ 12/25 \\ -9/25 \end{bmatrix}$$

De esta manera para nuestro ultimo vector tenemos

$$v_2 = a_3 - (q_1^T a_3) q_1 - (q_2^T a_3) q_2 = \begin{bmatrix} 14 \\ 4 \\ 2 \end{bmatrix} - (-4) \begin{bmatrix} 0 \\ -3/5 \\ -4/5 \end{bmatrix} - (-10) \begin{bmatrix} -20/25 \\ 12/25 \\ -9/25 \end{bmatrix} = \begin{bmatrix} 6 \\ 32/5 \\ -24/5 \end{bmatrix}$$

Asi nuestro ultimo vector es

$$q_3 = \frac{v_2}{\|v_2\|} = \frac{1}{10} \begin{bmatrix} 6 \\ 32/5 \\ -24/5 \end{bmatrix} = \begin{bmatrix} 3/5 \\ 16/25 \\ -12/25 \end{bmatrix}$$

De esta manera nuestra matriz ortogonal Q es

$$Q = (q_1 | q_2 | q_3) = \begin{bmatrix} 0 & -20/25 & 3/5 \\ -3/5 & 12/25 & 16/25 \\ -4/5 & -9/25 & -12/25 \end{bmatrix} = \begin{bmatrix} 0 & -20/25 & 15/25 \\ -15/25 & 12/25 & 16/25 \\ -20/25 & -9/25 & -12/25 \end{bmatrix}$$

Note que resulto ser la misma Q hallada por medio de HouseHolder, luego natrualmente tenemos que

$$R = Q^T A = \begin{bmatrix} 5 & -25 & -4 \\ 0 & 25 & -10 \\ 0 & 0 & 10 \end{bmatrix}$$

Asi la factorizacion $A = QR$ resulta ser la misma que en el punto anterior, es decir

$$\begin{bmatrix} 0 & -20 & 14 \\ -3 & 27 & 4 \\ -4 & 11 & 2 \end{bmatrix} = \begin{bmatrix} 0 & -20/25 & 15/25 \\ -15/25 & 12/25 & 16/25 \\ -20/25 & -9/25 & -12/25 \end{bmatrix} \begin{bmatrix} 5 & -25 & -4 \\ 0 & 25 & -10 \\ 0 & 0 & 10 \end{bmatrix}$$

c) Implemente en Matlab los métodos de ortogonalización de Gram-Schmidt y Householder, para calcular $A = QR$, compare los resultados numéricos con los encontrados en las partes (a) y (b).

Solución. Al implementar los métodos en Matlab, obtuvimos exactamente los mismos resultados calculados en las partes (a) y (b), por tanto podemos decir que para ésta matriz en específico, ambos métodos resultan estables numéricamente.

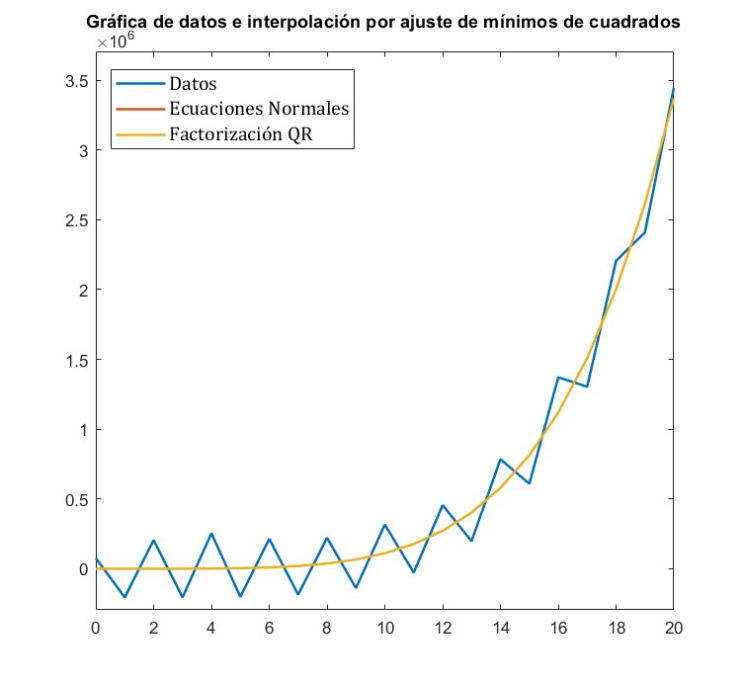
Ejercicio 2

Descargue el archivo `Datos.txt` de la página del curso. En este encontrará un conjunto de 21 datos. Copie estos datos y calcule el polinomio de ajuste de grado 5

$$p(x) = c_0 + c_1 x + c_2 x^2 + c_3 x^3 + c_4 x^4 + c_5 x^5$$

utilizando los métodos de ecuaciones normales y factorización QR. Compare sus resultados con los valores certificados $c_i = 1$ para $i = 0, 1, \ldots, 5$. Encuentre el residual $\|Ac - y\|_2$ en cada caso, así como la diferencia relativa con respecto a los valores certificados. Escriba sus conclusiones.

Solución. Al usar el método de ecuaciones normales, se obtienen los coeficientes $\overline{c_0} = 0{,}999999672214$, $\overline{c_1} = 1{,}000000693559$, $\overline{c_2} = 0{,}999999742520$, $c_3 = 1{,}000000034633$, $\overline{c_4} = 0{,}999999998066$ y $\overline{c_5} = 1{,}000000000038$ y una norma residual de $9{,}140802371783 \times 10^5$, ya que los valores certificados son $c_i = 1$ para $i = 0, 1, \ldots, 5$ el error relativo coincide con el error absoluto, de esta manera tenemos que $|\overline{c_0} - c_0| = 0{,}327785240838 \times 10^{-6}$, $|\overline{c_1} - c_1| = 0{,}693559258246 \times 10^{-6}$, $|\overline{c_2} - c_2| = 0{,}257479413678 \times 10^{-6}$, $|\overline{c_3} - c_3| = 0{,}034633125922 \times 10^{-6}$, $|\overline{c_4} - c_4| = 0{,}001933269655 \times 10^{-6}$ y $|\overline{c_3} - c_3| = 0{,}000038107517 \times 10^{-6}$, por lo que obtenemos una precisión de al menos 6 cifras decimales en cada coeficiente.
Ahora, si resolvemos el sistema usando factorización QR, nuestros coeficientes ahora son $\hat{c_0} = 1{,}000000001758$, $\hat{c_1} = 0{,}999999996403$, $\hat{c_2} = 1{,}000000001317$, $\hat{c_3} = 0{,}999999999824$, $\hat{c_4} = 1{,}000000000009$, $\hat{c_5} = 0{,}999999999999$, con una norma residual de $9{,}140802371783 \times 10^5$ y los errores correspondientes $|\hat{c_0} - c_0| = 0{,}000019029222 \times 10^{-8}$, $|\hat{c_1} - c_1| = 0{,}000972200098 \times 10^{-8}$, $|\hat{c_2} - c_2| = 0{,}017562828968 \times 10^{-8}$, $|\hat{c_3} - c_3| = 0{,}131761268562 \times 10^{-8}$, $|\hat{c_4} - c_4| = 0{,}359638396840 \times 10^{-8}$, y $|\hat{c_5} - c_5| = 0{,}175833236859 \times 10^{-8}$, en este caso obtuvimos una precisión de al menos 8 cifras decimales en cada coeficiente, lo cual quiere decir que éste método resulta mejor para resolver el sistema que las ecuaciones normales, aún así ambos métodos resultan ser muy precisos, pues los errores son despreciables. Al graficar el conjunto de datos y los dos polinomios obtenidos es practicamente imposible notar la diferencia entre ellos.



Ejercicio 3

Sean

$$A = \begin{bmatrix} a & 0 & 0 \\ a\delta & a & 0 \\ 0 & a\delta & a \end{bmatrix}, \quad a < 0, \delta > 0, \quad b = \begin{bmatrix} -1 \\ -1{,}1 \\ 0 \end{bmatrix}$$

a) Obtenga el número de condición de A.

Solución. Primero note que A es invertible ya que

$$\det A = a \det \begin{pmatrix} a & 0 \\ a\delta & a \end{pmatrix} = a^3$$

entonces como $a \neq 0$, podemos calcular el número de condición $\kappa_\infty(A) = \|A\|_\infty \|A^{-1}\|_\infty$, donde

$$A^{-1} = \frac{1}{a} \begin{pmatrix} 1 & 0 & 0 \\ -\delta & 1 & 0 \\ \delta^2 & -\delta & 1 \end{pmatrix}$$

Esto nos da que $\kappa_\infty(A) = a\|B\|_\infty \dfrac{1}{a} \|B^{-1}\|_\infty = \|B\|_\infty \|B^{-1}\|_\infty$, donde $B = \begin{pmatrix} 1 & 0 & 0 \\ \delta & 1 & 0 \\ 0 & \delta & 1 \end{pmatrix}$, así

$$\kappa_\infty(A) = \left\| \begin{pmatrix} 1 & 0 & 0 \\ \delta & 1 & 0 \\ 0 & \delta & 1 \end{pmatrix} \right\|_\infty \left\| \begin{pmatrix} 1 & 0 & 0 \\ -\delta & 1 & 0 \\ \delta^2 & -\delta & 1 \end{pmatrix} \right\|_\infty = (1+\delta)(1+\delta+\delta^2)$$

b) Estudie el condicionamiento del sistema $Ax = b$ en función de los valores de δ. Interprete su resultado.

Solución. En $\delta = 0$ el problema se comporta bien, hacemos este caso ya que aunque $\delta > 0$, hay que pensar en valores cercanos a 0, en estos el problema se comporta bien, note que el conjunto solución de $(1+\delta)(1+\delta+\delta^2) > 0$ es justamente $\delta > 0$, y esta función es creciente, entonces conforme δ se vuelve más grande, el número de condición aumenta y nuestro problema se vuelve sensible a errores de redondeo y perturbaciones en los datos. Más precisamente

- $\lim_{\delta \to 0} \kappa_\infty(A) = 1$.
- $\lim_{\delta \to \infty} \kappa_\infty(A) = \infty$

c) Si $a = -1$, $\delta = 0{,}1$ y se considera $x^* = (1, 9/10, 1)^T$ como solución aproximada del sistema $Ax = b$ (sin obtener la solución exacta), determine un intervalo en el que esté comprendido el error relativo. ¿Es coherente con la respuesta dada en el apartado anterior?

Solución. Reemplazando $a = -1, \delta = 0{,}1$ y tomando $x^* = (1, 9/10, 1)^T$ se sigue que

$$Ax^* = \begin{pmatrix} -1 & 0 & 0 \\ -0{,}1 & -1 & 0 \\ 0 & -0{,}1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 9/10 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ -1{,}09 \end{pmatrix} \approx \begin{pmatrix} -1 \\ -1{,}1 \\ 0 \end{pmatrix}.$$

Para determinar un intervalo del error podemos utilizar la fórmula

$$\frac{1}{\kappa_\infty(A)} \frac{\|b - b^*\|_\infty}{\|b\|_\infty} \leq \frac{\|x - x^*\|_\infty}{\|x\|_\infty} \leq \kappa_\infty(A) \frac{\|b - b^*\|_\infty}{\|b\|_\infty},$$

en efecto $\kappa_\infty(A) = (1 + 0{,}1)\left((0{,}1)^2 + 0{,}1 + 1\right) \approx 1{,}221$, $\|b\|_\infty = 1{,}1$ y $\|b - b^*\|_\infty = 1{,}09$. aplicando esto a la fórmula obtenemos

$$\frac{1}{1{,}211} \frac{1{,}09}{1{,}1} \leq \frac{\|x - x^*\|_\infty}{\|x\|_\infty} \leq 1{,}221 \frac{1{,}09}{1{,}1},$$

esto es

$$0{,}818 \leq \frac{\|x - x^*\|_\infty}{\|x\|_\infty} \leq 1{,}20.$$

Esto no es incoherente con el punto anterior ya que el punto anterior me dice que el sistema $Ax = b$ es bien condicionado para δ pequeño, en este caso $\delta = 0{,}1$, esto no significa que al introducir un x arbitrario el error deba ser pequeño, solo nos da que el sistema se comporta bien ante perturbaciones en los datos y errores de redondeo, sin embargo si me proporcionan un x erroneo como solución, esto no tiene nada que ver con lo antes mencionado, el error no necesariamente debe ser pequeño.

d) Si $a = -1$ y $\delta = 0{,}1$, ¿es convergente el método de Jacobi aplicado a la resolución del sistema $Ax = b$? Realice tres iteraciones a partir de $x_0 = (0, 0, 0)^T$.

Solución. Tomando a y δ como fueron dados en el enunciado tenemos que

$$A = \begin{bmatrix} -1 & 0 & 0 \\ -0{,}1 & -1 & 0 \\ 0 & -0{,}1 & -1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 \\ 0{,}1 & 0 & 0 \\ 0 & 0{,}1 & 0 \end{bmatrix} = D - L - U$$

Note que tenemos que $D = -I$ y que $U = 0$ en este caso, luego la matriz de iteración del método de Jacobi esta dada por

$$T_J = D^{-1}(L + U) = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 \\ 0{,}1 & 0 & 0 \\ 0 & 0{,}1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ -0{,}1 & 0 & 0 \\ 0 & -0{,}1 & 0 \end{bmatrix}$$

Luego note que $\|T_J\|_\infty = 0{,}1 < 1$, esto nos asegura que la sucesión dada por el método converge a la solución del sistema $Ax = b$. Ahora para realizar las tres iteraciones del

método, recordemos que la forma matricial del método de Jacobi esta dada por

$$x^{(k+1)} = T_J x^{(k)} + D^{-1} b$$

Como $D^{-1} = -I$, tenemos que

$$D^{-1} b = \begin{bmatrix} 1 \\ 1{,}1 \\ 0 \end{bmatrix}$$

Con esta información, consideremos la primera iteración

$$x^{(1)} = \begin{bmatrix} 0 & 0 & 0 \\ -0{,}1 & 0 & 0 \\ 0 & -0{,}1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 1{,}1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1{,}1 \\ 0 \end{bmatrix}$$

Ahora para la segunda iteración tenemos que

$$x^{(2)} = \begin{bmatrix} 0 & 0 & 0 \\ -0{,}1 & 0 & 0 \\ 0 & -0{,}1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1{,}1 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 1{,}1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -0{,}1 \\ -0{,}11 \end{bmatrix} + \begin{bmatrix} 1 \\ 1{,}1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ -0{,}11 \end{bmatrix}$$

Para finalizar, la tercera iteración esta dada por

$$x^{(3)} = \begin{bmatrix} 0 & 0 & 0 \\ -0{,}1 & 0 & 0 \\ 0 & -0{,}1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -0{,}11 \end{bmatrix} + \begin{bmatrix} 1 \\ 1{,}1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -0{,}1 \\ -0{,}1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1{,}1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ -0{,}1 \end{bmatrix}$$

En teoría $x^{(3)}$ es una solución aproximada del sistema, pero note que si efectuamos la multiplicación de esta aproximación con A tenemos que

$$Ax^{(3)} = \begin{bmatrix} -1 & 0 & 0 \\ -0{,}1 & -1 & 0 \\ 0 & -0{,}1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -0{,}1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1{,}1 \\ 0 \end{bmatrix} = b.$$

Note que justamente $x^{(3)}$ resulto ser no solo una aproximación, sino la solución exacta del sistema, es decir, que en este caso ademas de saber que el método de Jacobi converge, concluimos que convergía en tan solo 3 iteraciones.

Ejercicio 4

Sea $A \in \mathbb{R}^{n \times n}$. Probar que $\lambda = 1$ es un valor propio de la matriz de iteración del método de Jacobi (o Gauss-Seidel) de A si y solo si A no es invertible.

Demostración. Primero tomemos la matriz $A = D - L - U$, donde D es una matriz diagonal, L es estrictamente triangular inferior y U es estrictamente triangular superior. Probemos el resultado para el método de Jacobi.
($\Rightarrow$) Recordemos que la matriz de iteración esta dada por

$$T_J = D^{-1}(L + U).$$

Si $\lambda = 1$ es un valor propio de T_J, existe un vector no nulo v, tal que

$$T_J v = 1 \cdot v = v,$$

si reemplazamos tenemos que

$$D^{-1}(L+U)v = v,$$

multiplicando por D a izquierda a ambos lados

$$(L+U)v = Dv.$$

Pasando a sumar todo al lado derecho y distribuyendo obtenemos

$$0 = Dv + Lv + Uv.$$

Factorizando v a derecha

$$0 = (D - L - U)v.$$

Note que esta ultima linea dice que existe un vector no nulo v, que es solución de la ecuación $Ax = 0$, por lo que podemos concluir que A no es invertible.
($\Leftarrow$) Supongamos que A no es invertible, luego existe un vector no nulo v, tal que $Av = 0$, Si descomponemos A tenemos que

$$(D - L - U)v = Dv - Lv - Uv$$

Si despejamos para Dv

$$\begin{aligned} Dv &= Lv + Uv \\ &= (L+U)v \end{aligned}$$

Por ultimo multiplicando por D^{-1}, tenemos que

$$v = D^{-1}(L+U)v = T_J v.$$

Esto quiere decir que v es un vector propio para T_J con valor propio 1, así concluimos que la matriz de iteración de Jacobi tiene valor propio $\lambda = 1$.
La prueba para la matriz de iteración de Gauss-Seidel se realiza de manera similar.
($\Rightarrow$) Recordemos que la matriz de iteración para este caso es

$$T_{GS} = (D - L)^{-1}U.$$

Nuevamente por la definición de valor propio, existe un v no nulo tal que

$$(D - L)^{-1}Uv = v.$$

Multiplicando por $D - L$ tenemos que

$$Uv = (D - L)v,$$

Si pasamos a restar y factorizamos v tenemos que

$$0 = (D - L)v - Uv = (D - L - U)v.$$

Así hemos encontrado que v, es un vector no nulo que soluciona $Ax = 0$. Concluyendo así que A no es invertible.
($\Leftarrow$) Supongamos que A no es invertible, luego existe v no nulo tal que $Av = 0$, así por como descomponemos A se tiene que $Av = (D - L - U)v = (D - L)v - Uv$. Como esta igualado a 0, Sumamos Uv obteniendo así

$$(D - L)v = Uv.$$

Multiplicando por la inversa de $D - L$ llegamos a que $v = T_{GS}v$, mostrando así que v es un vector propio para T_{GS}, con valor propio $\lambda = 1$. Concluyendo así que el hecho es cierto para ambas matrices de iteración.

∪˰∪

Ejercicio 5

Considere el sistema $Ax = b$ donde

$$A = \begin{bmatrix} 3 & -1 & -1 & 0 & 0 \\ -1 & 4 & 0 & -2 & 0 \\ -1 & 0 & 3 & -1 & 0 \\ 0 & -2 & -1 & 5 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ -26 \\ 3 \\ 47 \\ -10 \end{bmatrix}$$

a) Investigue la convergencia de los métodos de Jacobi, Gauss-Seidel y sobrerelajación.

Solución. Para determinar la convergencia del método de Jacobi basta observar que la matriz A es estrictamente diagonal-dominante por filas, por lo tanto el método de Jacobi converge. Al programar el método y con una tolerancia de 0.001 respecto al error en $\|\cdot\|_\infty$ se observa que el método se detiene luego de 28 iteraciones y se obtiene un error de aproximadamente $7{,}7359 \times 10^{-4}$.

Veamos ahora que A es simétrica definida positiva, pues así garantizamos la convergencia del método de sobrerelajación para $\omega \in (0, 2)$ y por lo tanto del método de Gauss - Seidel. Procederemos a calcular $x^T Ax$ para un vector no nulo arbitrario:

$$
\begin{aligned}
x^T Ax &= \begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 \end{bmatrix} \begin{bmatrix} 3 & -1 & -1 & 0 & 0 \\ -1 & 4 & 0 & -2 & 0 \\ -1 & 0 & 3 & -1 & 0 \\ 0 & -2 & -1 & 5 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} \\
&= \begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 \end{bmatrix} \begin{bmatrix} 3x_1 - x_2 - x_3 \\ -x_1 + 4x_2 - 2x_4 \\ -x_1 + 3x_3 - x_4 \\ -2x_2 - x_3 + 5x_4 - x_5 \\ -x_4 + 2x_5 \end{bmatrix} \\
&= 3x_1^2 + 4x_2^2 + 3x_3^2 + 5x_4^2 + 2x_5^2 - 2x_1x_2 - 2x_1x_3 - 4x_2x_4 - 2x_3x_4 - 2x_4x_5 \\
&= (x_1 - x_2)^2 + (x_1 - x_3)^2 + 2(x_2 - x_4)^2 + (x_3 - x_4)^2 + (x_4 - x_5)^2 + x_1^2 + x_2^2 + x_3^2 + x_4^2 + x_5^2 \\
&> 0
\end{aligned}
$$

Con esto vemos que $x^T Ax > 0$ y por lo tanto A es definida positiva, así, los métodos de sobrerelajación con $\omega \in (0,2)$ y de Gauss-Seidel convergen. Al programar el método de Gauss-Seidel y fijar la misma tolerancia que con el método de Jacobi, éste se detiene luego de 14 iteraciones y un error de aproximadamente $6{,}8814 \times 10^{-4}$, con lo cual notamos que el método converge significativamente más rápido que el de Jacobi. Ahora, si programamos el método de sobrerelajación tomando $\omega = 1{,}18$ usando las mismas condiciones de parada anteriores, éste se detiene luego de 9 iteraciones, por tanto podemos decir que de los tres métodos, es el que converge más rápido.

b) ¿Cuál es el radio espectral de la matriz J y de la matriz S?

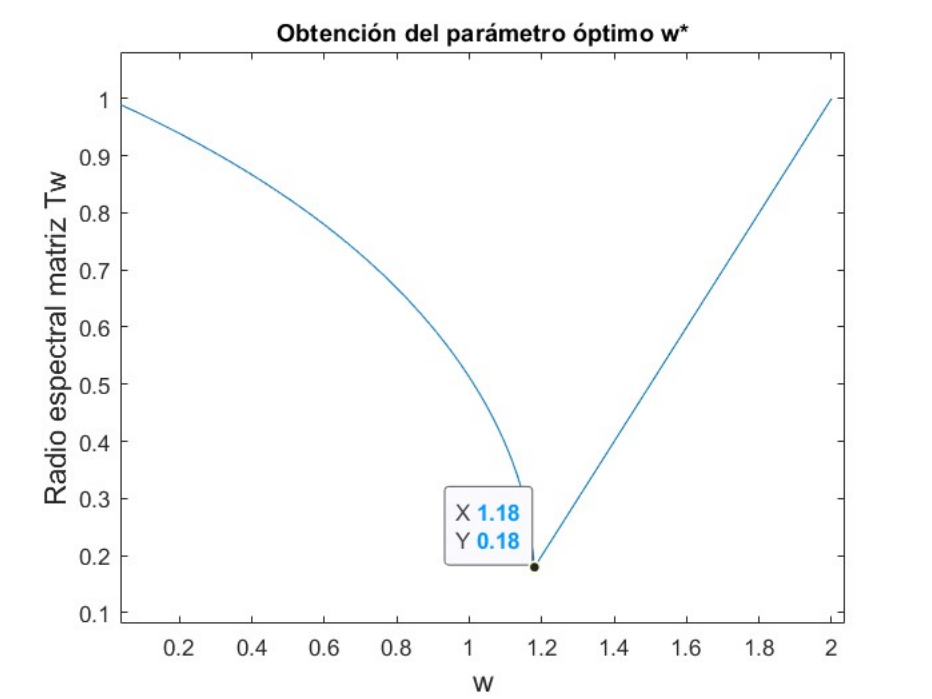
Solución. Para esto primero calculamos las matrices T_J y T_{GS} en Matlab de acuerdo a las fórmulas $T_J = D^{-1}(U+L)$ y $T_{GS} = (D-L)^{-1}U$ y luego calculamos el radio espectral usando el siguiente código

```
T_J=inv(D)*(L+U);
T_GS=inv(D-L)*U;
p_J=max(abs(eig(T_J))); %radio espectral de T_J
p_GS=max(abs(eig(T_GS))); %radio espectral de T_GS
```

así obtuvimos que $\rho(T_J) = 0{,}7158$ y $\rho(T_{GS}) = 0{,}5123$, observe que $\rho(T_{GS}) < \rho(T_J)$, lo cual es esperable pues el radio espectral determina la velocidad de convergencia de cada método.

c) Aproxime con dos cifras decimales el parámetro de sobrerelajación ω^*.

Solución. Teniendo en cuenta que el radio espectral determina la velocidad de convergencia, podemos calcular el radio espectral de la matriz de iteración, variando ω entre 0 y 2, con pasos de 0,01 para obtener la precisión deseada, luego escogemos el ω que minimice el radio espectral de la matriz T_ω. En este caso obtuvimos que el parámetro ω^* es aproximadamente 1.18, como se observa en la gráfica:



d) ¿Qué reducción en el costo operacional ofrece el método de sobrerelajación con el parámetro ω^*, en comparación con el método de Gauss-Seidel?

Solución. La reducción en el costo operacional puede interpretarse como la diferencia en el número de iteraciones de cada método, pues en general cada iteración tiene el mismo número de operaciones, sin tener en cuenta el cálculo de las matrices de iteración, pues éste se puede realizar antes de implementar cada método. En ese caso, programamos ambos métodos variando la tolerancia al error, de tal manera que se pudiera evidenciar la diferencia entre las iteraciones de cada método. De esta forma obtenemos la siguiente tabla:

Tolerancia	1	10^{-1}	10^{-2}	10^{-3}	10^{-4}	10^{-5}	10^{-6}	10^{-7}	10^{-8}	10^{-9}	10^{-10}
Sobrerelajación	4	6	7	9	10	11	12	14	15	16	17
Gauss-Seidel	4	8	11	15	18	22	25	28	32	35	39

Aquí observamos que a medida que la tolerancia se hace más pequeña, la relación entre la cantidad de iteraciones de cada método es aproximadamente 0.43, lo cual significa que con el metodo de Sobrerelajación reducimos la cantidad de operaciones en un 56% con respecto al método de Gauss-Seidel.

e) ¿Cuántas iteraciones más requiere el método de Gauss-Seidel para lograr una precisión mejorada en una cifra decimal? ¿Cuántas necesita el método de sobrerelajación con ω^*?

Solución. Si observamos nuevamente la tabla del inciso anterior, variamos la tolerancia justamente para que cada vez se mejore la precisión en una cifra decimal, y notamos que para el método de Gauss-Seidel se necesitan al rededor de 3 o 4 iteraciones más para lograr esto. En cambio, para lograr esta misma mejoría en la precisión con el método de

sobrerrelajación, basta iterar 1 o 2 veces más, lo cual también indica que éste método converge mucho más rápido que el anterior.

Ejercicio 6

El operador Laplaciano en 2D se define como:

$$\nabla^2 u(x,y) = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$$

donde $u(x,y)$ es una función que representa la intensidad de los pixeles de la imagen en el dominio espacial. El operador Laplaciano se utiliza para detectar bordes porque responde a las variaciones de la intensidad de los pixeles en la vecindad de cada punto. En áreas de la imagen donde la intensidad varía rápidamente (bordes), el Laplaciano tiene un valor alto, mientras que en áreas homogéneas (sin bordes) el Laplaciano es cercano a cero. El proceso de detección de bordes implica calcular el Laplaciano de la imagen $u(x,y)$, lo que da como resultado un mapa de bordes.
En este ejercicio estamos interesados en reconstruir la imagen original $u(x,y)$ a partir de los bordes detectados. Para esto debemos resolver la ecuación de Poisson en 2D

$$\nabla^2 u(x,y) = f(x,y)$$

donde $f(x,y)$ es el mapa de bordes. El problema (1) puede ser discretizado en un sistema de ecuaciones lineales de la forma:

$$Au = f$$

donde A es una matriz que representa el operador Laplaciano discreto, u es el vector que contiene los valores de la imagen original en cada pixel (a reconstruir), y f es el vector que contiene los valores de los bordes detectados.

Solución. Lo primero que debemos hacer es discretizar el laplaciano, para esto usamos el teorema de Taylor. A saber

$$u(x+h,y) = u(x,y) + hu_x(x,y) + \frac{h^2\partial_x^2 u(x,y)}{2} + O(h^3),$$
$$u(x-h,y) = u(x,y) - hu_x(x,y) + \frac{h^2\partial_x^2 u(x,y)}{2} + O(h^3),$$

realizando esto para $u(x,y+h)$ y $u(x,y-h)$ obtenemos que

$$\Delta u(x,y) \approx \frac{u(x-h,y) + u(x+h,y) + u(x,y-h) + u(x,y+h) - 4u(x,y)}{h^2}.$$

Para reconstruir la imagen debemos encontrar $u(x,y)$, la función que nos da la intensidad de cada pixel, esto es, resolver la ecuación de Poisson $\Delta u(x,y) = f(x,y)$ donde $f(x,y)$ es el mapa de bordes. Primero consideremos para el problema una malla regular, tomaremos $h = 1$ ya que cada pixel representa un paso, en una dimensión la matriz que obtenemos es tridiagonal, sin embargo al discretizar la ecuación de Poisson en 2D debemos mover f en coordenadas x y y, por lo tanto debemos analizarlo con detalle.

Como usamos usa malla regular de $n \times n$ (porque el problema es 2D), cada fila de la matriz sería de $n \times n$, obteniendo

$$\Delta u_{ij} = u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{ij} = f_{ij}$$

donde $2 \leq i \leq m-1$ y $2 \leq j \leq n-1$. Esto da lugar a un sistema lineal de dimensión $n^2 \times n^2$, $Au = f$, donde

$$A = \begin{bmatrix} D & I & 0 & 0 & 0 & \cdots & 0 \\ I & D & I & 0 & 0 & \cdots & 0 \\ 0 & I & D & I & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & I & D & I & 0 \\ 0 & \cdots & \cdots & 0 & I & D & I \\ 0 & \cdots & \cdots & \cdots & 0 & I & D \end{bmatrix}, \text{ donde } D = \begin{bmatrix} -4 & 1 & 0 & 0 & 0 & \cdots & 0 \\ 1 & -4 & 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & -4 & 1 & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 1 & -4 & 1 & 0 \\ 0 & \cdots & \cdots & 0 & 1 & -4 & 1 \\ 0 & \cdots & \cdots & \cdots & 0 & 1 & -4 \end{bmatrix},$$

y como f es una matriz, se debe aplanar como vector columna. Por ejemplo en 3×3

$$A = \left[\begin{array}{ccc|ccc|ccc} -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -4 & 0 & 0 & 1 & 0 & 0 & 0 \\ \hline 1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1 \\ \hline 0 & 0 & 0 & 1 & 0 & 0 & -4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 \end{array}\right].$$

Ahora debemos resolver los sistemas implementando los métodos de Jacobi,Gauss-Seidel y SOR, esto lo hicimos en Matlab y el código es extenso, por lo que lo adjuntamos en el cuaderno de Jupyter, sin embargo vamos a ver aquí los resultados obtenidos.

Para el método de Jacobi aprovechamos que la inversa de la matriz diagonal es simplemente invertir la diagonal, a saber $1/a_{ii}$, luego el código es bastante eficiente y puede correr 100 iteraciones muy rápido.

En Gauss-Seidel no fue posible implementar el método de manera matricial, invertir matriz $(D-L)$ es un trabajo costoso teniendo en cuenta que la primera imagen es de 240×240, es decir una matriz de 57600×57600, una sola iteración costaba varios minutos. En vista de esto optamos por implementar el método resolviendo cada ecuación y sin definir la matriz A, usando claramente que

$$\Delta u_{ij} = u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{ij} = f_{ij},$$

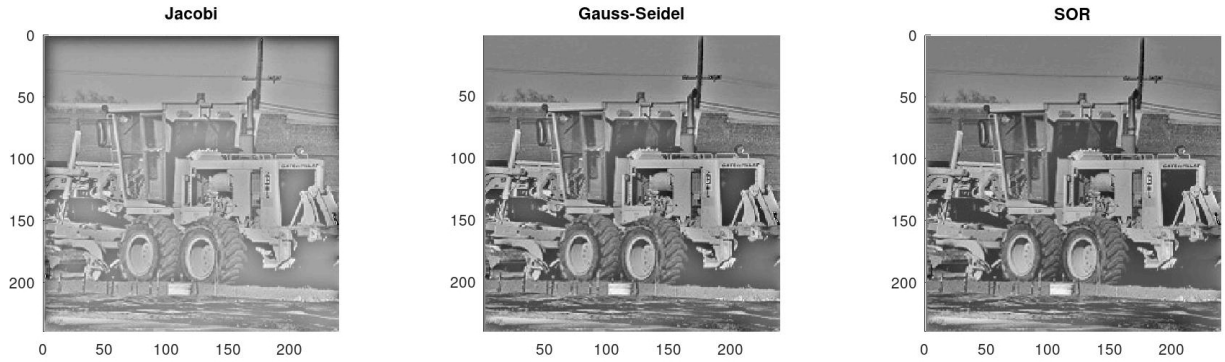
se obtiene una fórmula eficiente para hace Gauss-Seidel, a saber

$$u_{ij}^{(k+1)} = \frac{1}{4}\left(\left(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k+1)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k+1)}\right) - f_{ij}\right),$$

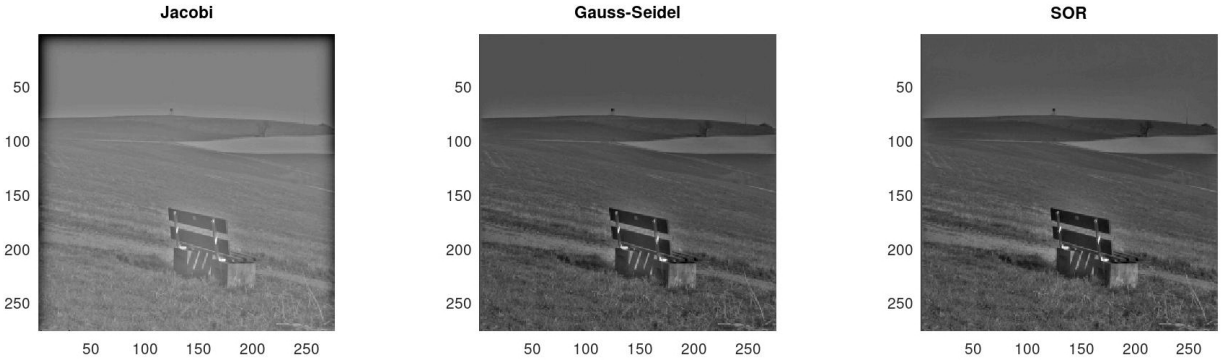
esto nos permitió implementar todo de manera eficiente. Para SOR la idea fue exactamente la misma pero aplicando la relajación

$$u_{ij}^{k+1} = (1-\omega)u_{ij}^{k} + \frac{\omega}{4}\left(\left(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k+1)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k+1)}\right) - f_{ij}\right),$$

en este caso tomamos $\omega = 1{,}5$ ya que la idea era acelerar la convergencia, esto se logra con $1 < \omega < 2$. Finalmente para la primera imagen, dada por el archivo Bordes1, obtuvimos los siguientes resultados aplicando 100 iteraciones.



Para la segunda imagen también aplicamos 100 iteraciones, obtuvimos



Aplicamos 100 iteraciones para que se observara diferencia entre los métodos ya que con muchas iteraciones el problema se estabiliza de manera que no notaremos diferencia entre aplicar un método y otro.